Laborator 08

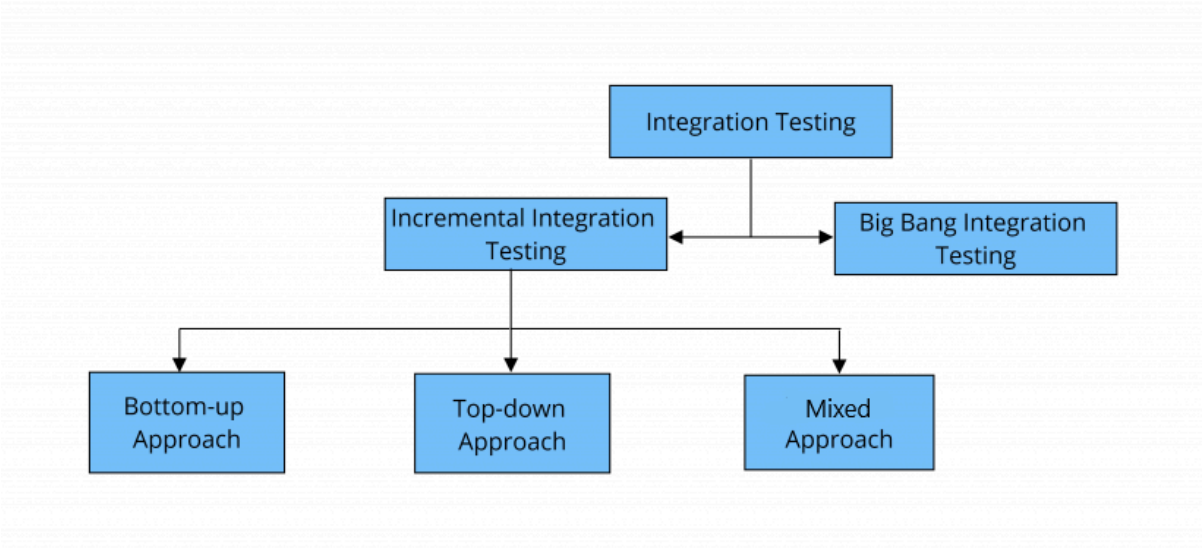
Testarea de integrare

Testarea de integrare este un tip de testare în care două sau mai multe componente diferite ale unei aplicații software sunt testate împreună pentru a valida corectitudinea interacțiunii dintre acestea. Testarea de integrare este realizată de obicei după testarea unitară și ajută la identificarea timpurie a defectelor aplicației pe parcursul ciclului de dezvoltare.

Strategii de testare de integrare

Există 4 strategii principale de testare de integrare:

1. Big-Bang
2. Bottom-Up
3. Top-Down
4. Mixed



Big-Bang

Strategia “Big Bang” constă în testarea simultană a tuturor componentelor aplicației, astfel încât să se obțină un sistem complet. Scopul acestei strategii este de a verifica funcționarea aplicației ca întreg și de a identifica defecte apărute din interacțiunea tuturor modulelor. Această abordare prezintă un efort mai redus de planificare, dar este practică doar pentru sisteme de dimensiuni și complexitate mică, fiind dificilă izolarea și diagnosticarea tuturor tipurilor de erori.

Bottom-Up

Strategia “Bottom-Up” constă în testarea incrementală a aplicației, astfel încât testarea modulelor de nivel inferior precede testarea modulelor de nivel superior. În acest tip de abordare se testează interacțiunea dintre module ale aplicației, grupate în clustere, pornind de la modulele cele mai de jos, peste care se adaugă incremental module de la nivele superioare, până ce se obține o testare completă a sistemului. Strategia “Bottom Up” necesită ca modulele inferioare să fie complete înainte de a se începe testarea de integrare, ceea ce întârzie testarea sistemului în procesul de dezvoltare, dar aduce o eficacitate sporită a detectării defectelor.

Top-Down

Strategia “Top-Down” constă în testarea incrementală a aplicației, astfel încât testarea modulelor de nivel superior precede testarea modulelor de nivel inferior. Modulele aplicației sunt grupate și testate de sus in

jos, până ce se obține o testare completă a sistemului. Aceasta abordare nu necesită ca modulele inferioare să fie complete, funcționalitatea acestora fiind imitată de module stub, ceea ce permite ca testarea să poată începe devreme în ciclul de dezvoltare, însă vine cu dezavantajul că trebuiesc create un număr mare de module stub.

Mixed

Strategia “Mixed”, sau “Sandwiched” este o combinație a abordărilor “Top-Down” și “Bottom-Up”. În această abordare, focusul este axat pe testarea unui layer al aplicației, integrând atât module de nivel superior, cât și module de nivel inferior.

Exerciții

Setup

```
pip install requests
```

În partea practică vom realiza teste de integrare pentru o aplicație web de tip **REST API** care permite gestionarea task-urilor personale, fiind disponibilă aici: https://todo.pixegami.io [https://todo.pixegami.io]. Un task este caracterizat de un user id, task id, descriere (*content*) și status de finalizare (*is_done*).

În documentația aplicației (https://todo.pixegami.io/docs [https://todo.pixegami.io/docs]) sunt descrise operațiile expuse, și anume:

- crearea unui task
- obținerea unui task
- listarea task-urilor
- modificarea unui task
- ștergerea unui task

După ce a fost realizată testarea unitara a modulelor aplicației (endpoints, baze de date, s.a.), trebuie să fie validată interacțiunea dintre acestea. Astfel, se dorește să se testeze ca operațiile pe task-uri descrise mai sus se execută cu succes.

- Implementați un test de integrare pentru operația de creare a unui task.

Hint: validarea creării unui task presupune 2 request-uri, unul de **PUT** și altul de **GET**, testând interacțiunea dintre clientul web, endpoint-ul web și baza de date în care se stochează task-urile.

Verificare: validați că request-urile întorc codurile de status **HTTP** corecte (**200** în cazul lui **PUT** și **GET**) și că body-ul răspunsului primit este cel așteptat (în cazul lui **GET**, json-ul din body-ul primit ca răspuns trebuie să conțină task-ul introdus prin acțiunea **PUT**: *content*, *user_id*, *is_done*).

Atenție: task_id-ul este generat random server-side, astfel că task-id-ul trimis prin operația de **PUT** in create_task este irelevant, iar răspunsul request-ului create_task va conține task-id-ul generat.

- Implementați un test de integrare pentru operația de modificare a unui task.

- Implementați un test de integrare pentru operația de listare a 3 task-uri.

Hint: Creați 3 task-uri sub același user_id, apoi validați că operația de listare întoarce 3 task-uri.

Sugestie: Încercați să generați user_id-uri unice, altfel există șansa ca operația de listare să vă întoarcă task-uri create de colegii voștri. Pentru a genera id-uri unice, puteți folosi modulul de python **uuid**.

- Implementați un test de integrare pentru operația de ștergere a unui task.

Hint: Trebuie să validați că după operația de ștergere, un task nu poate fi obținut prin **GET**.